Trabajo Práctico Integrador — Parte 1

Redmine 7.0.3 sobre Vagrant, instalado con Ansible

Santiago Marcos

6 de noviembre de 2026

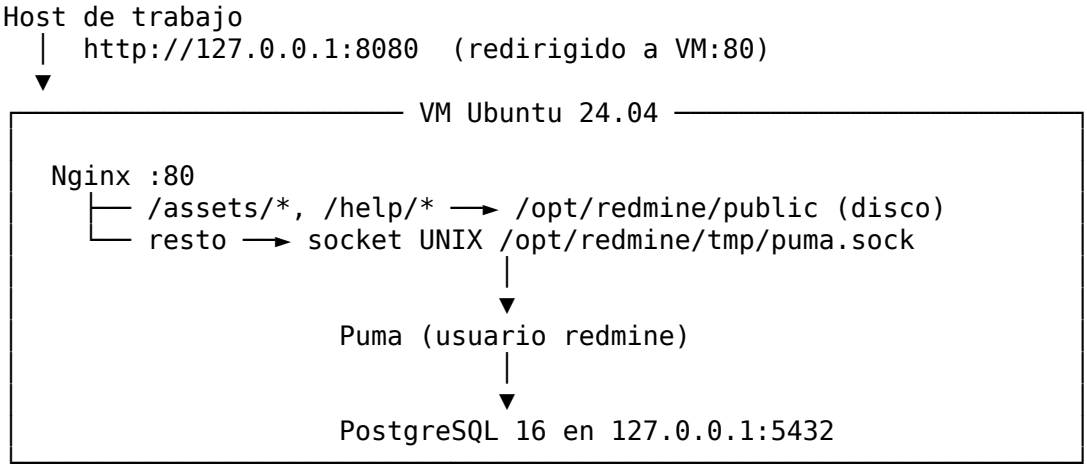
1. Objetivo y alcance

El objetivo de esta parte es instalar la última versión estable de **Redmine** (**7.0.3**) dentro de una caja Vagrant con **Ubuntu 24.04**, usando Ansible de forma declarativa. La entrega incluye roles, playbooks, inventario, verificación, pruebas con Molecule, linters y documentación.

Los requisitos concretos y su cumplimiento son:

Requisito	Solución implementada
Ruby propia, no la del sistema	Ruby **4.0.9** compilada con **rbenv** y `ruby-build`.
Usuario distinto de root	Usuario de sistema `redmine`; Puma corre con `User=redmine`.
Application server + web server	**Puma** por socket UNIX detrás de **Nginx** como proxy inverso.
Estáticos fuera de Ruby	Nginx sirve `/assets`, `/help`, favicon y páginas de error desde disco.
Base de datos soportada y migrable	**PostgreSQL 16** local, con conexión parametrizada.
Reinicio sin pérdida de servicio	Servicios `enabled`, `Restart=always` y orden de arranque en systemd.

2. Arquitectura



Redmine se instala en `/opt/redmine`. El home `/home/redmine` contiene el toolchain de Ruby (`rbenv`), no la aplicación. La base queda en la misma VM, pero la aplicación la consume por host, puerto y credenciales, de modo que puede moverse a otro servidor sin cambiar código.

3. Cumplimiento de los requisitos

3.1 Ruby instalada fuera del sistema operativo

El rol `redmine_ruby` clona rbenv y el plugin `ruby-build` en `/home/redmine/.rbenv`, compila Ruby **4.0.9** y fija esa versión como global. No se instala el paquete `ruby` de Ubuntu. La unidad systemd usa `rbenv exec bundle exec puma`, por lo que el proceso toma la Ruby de rbenv.

Verificación real dentro de la VM:

```
$ sudo -u redmine -H bash -lc '...; ruby -v'
ruby 4.0.9 (2026-10-23 revision 3e93b23afc) +PRISM [x86_64-linux]

$ command -v ruby
no instalado
```

Ruby 4.0.9 es la última estable de la serie 4.0 y Redmine 7.0 recomienda 4.0.4 o superior. Se evita además la regresión de rendimiento de wiki presente en Ruby 4.0.0–4.0.3.

3.2 Usuario no root

`redmine_common` crea el grupo y el usuario de sistema `redmine`, con home propio y permisos `0750`. La unidad `/etc/systemd/system/redmine.service` declara:

```
User=redmine
Group=redmine
WorkingDirectory=/opt/redmine
Restart=always
```

La verificación comprueba que existan procesos Puma del usuario `redmine` y que **no** exista ningún Puma ejecutado por `root`.

3.3 Puma detrás de Nginx y estáticos servidos por Nginx

Puma no abre puertos TCP: escucha en un socket UNIX con permisos de grupo (`umask=0007`). Nginx recibe el tráfico en el puerto 80 y actúa como proxy inverso:

```
upstream redmine_app {
    server unix:/opt/redmine/tmp/puma.sock fail_timeout=0;
}
location /assets/ {
    alias /opt/redmine/public/assets/;
    expires 30d;
    try_files $uri =404;
}
location / {
    proxy_pass http://redmine_app;
}
```

Además se activa en el archivo `production.rb` la opción `public_file_server.enabled`, condicionada a `RAILS_SERVE_STATIC_FILES`. Como esa variable no se define, Rails no sirve `/public` por su cuenta. La prueba pide el mismo asset directamente a Puma por el socket y espera un **404**; por Nginx espera **200** con `Server: nginx` y sin `X-Runtime`.

3.4 PostgreSQL local y preparado para migrar

`redmine_database` instala PostgreSQL 16, crea el rol y la base con `community.postgresql`, escucha solo en `localhost` y configura `scram-sha-256` en `pg_hba.conf` para las CIDR permitidas.

La conexión de Redmine se genera desde `config/database.yml.j2` con `redmine_db_host`, `redmine_db_port`, `redmine_db_name`, `redmine_db_user` y `redmine_db_password`. Para mover la

base alcanza con exportar/restaurar la base, copiar `/opt/redmine/files` si hay adjuntos y ajustar el host y el CIDR en el inventario. No hay cambios de código.

3.5 Reinicio de la VM

`postgresql`, `redmine` y `nginx` quedan enabled. Puma tiene `Restart=always` y Nginx arranca después de Redmine mediante el drop-in:

```
[Unit]
After=network.target redmine.service
Wants=redmine.service
```

Se comprobó con `vagrant reload`: la VM volvió a arrancar y Redmine respondió HTTP 200 sin intervención manual.

4. Roles y playbooks

Rol	Responsabilidad
`redmine_common`	Paquetes, locale, zona horaria, usuario/grupo y `/opt/redmine`.
`redmine_database`	PostgreSQL, rol, base, `pg_hba.conf` y parámetros de escucha.
`redmine_ruby`	rbenv, ruby-build, Ruby 4.0.9 y bundler.
`redmine`	Descarga, checksum, configuración, migraciones, assets, Puma y systemd.
`redmine_nginx`	Proxy inverso, estáticos, validación y orden de arranque.

El playbook `playbooks/site.yml` ejecuta los cinco roles en orden. La verificación reutiliza `roles/redmine/tasks/verify.yml` y comprueba servicios, usuario, Ruby, base, HTTP y estáticos.

El rol `redmine` se divide en `download.yml`, `configure.yml`, `database.yml`, `assets.yml` y `service.yml` para mantener el flujo legible. La descarga valida el SHA256 publicado; `unarchive` usa `creates: Gemfile`; `bundle check` decide si instalar gemas; las migraciones y `host_name` se ejecutan de forma controlada; y los assets se precompilan solo si faltan o si se fuerza con la variable `redmine_recompile_assets`.

5. Flujo Vagrant

El `Vagrantfile` usa la box `mw-ubuntu-24.04`, reenvía `127.0.0.1:8080` al puerto 80 de la VM y ejecuta el provisionador `ansible_local`. Dentro de la VM se instalan las colecciones de `requirements.yml` y se corre `playbooks/site.yml` con `--become`.

```
make lab-secret          # genera una contraseña local fuera de Git
vagrant up --provision
```

Si los puertos por defecto están ocupados por otra VM:

```
REDMINE_SSH_PORT=2223 REDMINE_HTTP_PORT=8081 vagrant up --provision
```

El inventario local (`inventory/hosts.yml`) usa `ansible_connection: local`, por lo que el playbook se ejecuta dentro de la VM. La contraseña de la base se toma de `vault_redmine_db_password`, con `make lab-secret` para el laboratorio o Ansible Vault para un entorno real.

6. Verificación y evidencias

Comprobación	Resultado
`systemctl is-active redmine nginx postgresql`	`active active active`
`systemctl is-enabled ...`	`enabled enabled enabled`
Ruby	`4.0.9` mediante rbenv; Ruby del sistema ausente
Puma	`8.0.2`, workers del usuario `redmine`
Nginx	`1.24.0 (Ubuntu)`
`playbooks/verify.yml`	`ok=21`, `failed=0`
Segunda ejecución de `site.yml`	`ok=60`, `changed=0`
`vagrant reload` + HTTP	Servicios activos; HTTP 200
Asset por Nginx	`200`, `Server: nginx`, `Cache-Control: public, immutable`
Asset directo a Puma	`404` (Ruby no sirve estáticos)

Resumen real del playbook de verificación:

```
PLAY RECAP
redmine : ok=21 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Salida real del asset servido por Nginx:

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Expires: Sun, 06 Dec 2026 14:32:43 GMT
Cache-Control: public, immutable
```

7. Calidad: linters, Molecule y documentación

Se ejecutaron las siguientes comprobaciones:

Herramienta	Resultado
`yamllint .`	Sin errores
`ansible-lint` (perfil production)	0 fallos, 0 warnings
`molecule test`	create → converge → idempotence → verify → destroy

El escenario Molecule usa Docker con systemd, Ubuntu 24.04, los cinco roles y las mismas verificaciones del playbook. Durante la revisión se corrigió el job de CI de Molecule para que instale las colecciones (`community.postgresql` y `community.general`) antes de ejecutar el escenario, y se declararon esas colecciones en `molecule/default/requirements.yml` para que la prueba sea autocontenida.

La documentación se revisó contra el código y la VM real. Se actualizaron `README.md` y `flujo.md` para reflejar el estado verificado, los puertos alternativos y el manejo de colecciones en Molecule. También se amplió `.gitignore` para evitar que vuelvan al repositorio los artefactos generados y los secretos locales.

En el rol `redmine` se agregó `notify: Reiniciar Redmine` a las plantillas de `database.yml`, `puma.rb` y `Gemfile.local`, para que un cambio de host/puerto de base o de workers se aplique efectivamente en el servicio.

8. Conclusión

La instalación cumple todos los puntos pedidos: Redmine 7.0.3 sobre Ubuntu 24.04, Ruby 4.0.9 fuera del sistema, usuario `redmine`, Puma detrás de Nginx con los estáticos servidos por Nginx, PostgreSQL

16 local y persistencia tras el reinicio. El flujo es idempotente, está verificado dentro de la VM y queda automatizado con roles, playbooks, Molecule y linters.